

sudo afw .recloud

Data Science

• Evolution of Technology

Telephone — Mobile — Desktop — Cloud

To analyze complex data we used various algorithms & tools & technologies

• Data Source : IoT

Collection of interconnected devices that communicate and transfer data through the Internet.

As per Cisco recent survey, IoT is generating more than 500 Zettabyte of data per year (predicted).

• Data Source : Social Media

Posts on fb, Insta, Email

• Data Source : Other Factor

• Banking / finance • Healthcare • Media & Entertainment
• Education • Transportation

Category of data

- Structured data (Easy to search, Text-based)
Pre-defined Data Model (phone no, date, Customer no)
Store in database (Data Mart, Data Warehouse)

Applications :-

Inventory control, Airline reservation system

- Semi-Structured Data

Loosely organized can be in html, xml or json format.

Example (Server logs, Tweets organized by hashtags, Email sorting by folder)

Resides in (store) Files with tagged-text format.

Applications: Server logs, sensor output

- Unstructured Data

Not in pre-defined format & difficult to search within. (documents, images, audio/video)

Example (Sensor data, Surveillance imagery)

Resides in (Data Lake, MS Azure)



Database
Storage

Application (audio, video call on social media, tools for editing)

What is Data Science?

Data Science is an inter-disciplinary field that uses scientific methods, processes algorithms and system to extract knowledge and insights from structured and unstructured data related to business problem.

Factors making Data Science Ubiquitous

- Increasing no. of connected devices (Internet)
- Decreasing Internet Transit Pricing (cheaper) ^{for data trans}
- Decreasing Costs for Data Storage
- Decreasing computational costs

Applications of Data Science

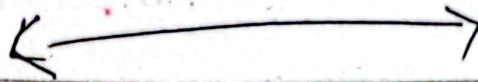
- 1 - Social Media (Recommendation system, Ad placement, Sentiment analysis, E-mail filtering)
- 2 - Banking (Fraud Detection, Credit scoring, Price optimization, Anti-money Laundering)
- 3 - E-commerce (RS, Upselling, Cross-selling, Discount Price optimization, Business forecasting (manage demand))
- 4 - Search Engines (Personalized search result, Ad placement, Search algorithm)

5- Travel (Predict Flight Delay, Dynamic pricing, Best route selection)

6- Healthcare (Medical imaging (x-ray, ultrasound) Disease prediction, ~~Self-driving~~)

7- Automation (self-driving cars, Robots)

Process real-time images/video perform object detection, tracking, segmentation



Efficiency: Minimize data loss during data transmission.

Automation: Provide a way to use resources efficiently.

Confusion matrix

Accuracy, Precision, Recall, F1 score

Dataset:

Dataset download from Kaggle

using csv file perform operation like read, write etc

Create record in database

connect with python

DataFrame

Python

→ File extension → .py

→ Case-sensitive

For output

```
print("Aslamu Mulkum")
```

single comma
also used

→ Variable

```
name = "Usman"
```

```
age = 22
```

```
is-adult = True
```

```
CGPA = 2.4
```

→ For Input

```
input("What is your age")
```

→ Type-Conversion

```
num1 = input("Enter first no.")
```

```
num2 = input("Enter second no.")
```

```
sum = int() num1 + int() num2
```

```
print(sum)
```

For type conversion we also use

— `str()`

— `float()`

— `bool()`

→ Operation on String

name = "Spiderman"

print (name.upper())

print (name.find("m"))

→ find value
in string
at index

print (name.replace("S", "1S"))

print ("man" in name)

→ bool function
told value is
exist or not

print (name.len())

→ told length
of string

→ Arithmetic Operator

i = 5

// 5

i += 2

// 7

or

i = i + 2

// 7

For Square

print (5 ** 2)

print (5 // 2)

for floor
value
modulus

print (5 % 2)

Precedence Concept

result = 2 + 3 * 5

print (result)

// 17

→ Comments

Calculate Sum

→ Comparison Operator ($>$, $<$, $<=$, $>=$, $==$, $!=$)

$2 > 3$, $3 < 2$, $3 >= 2$, $3 <= 3$, $4 == 4$
greater than, less than, greater than equal to, less than equal to, equal to

$3 != 3$ (Not equal to)

→ Logical Operator (and, or, not)

E.g. `print(2 > 3 or 2 > 1)` // True

`print(2 > 3 and 2 > 1)` # False

`print(not 2 == 3)` # True

→ Conditional Statement

1) If-statement

age = 19

if age $>=$ 18 :

`print("You are an adult")`

`print("Now, You can drive car")`

`print("Thanks")`

→ tell now we write block of statement

space in two statement tells they are part of if

2) elif-statement (else-if)

elif age $<$ 18 and age $>$ 3 :

`print("you are not an adult")`

3) else-statement

else:

`print("You are child")`

Mini Project (Calculator)

```
num1 = input("Enter first number :")
operator = input("Enter operator(+, -, /, *, %):")
num2 = input("Enter second number :")
num1 = int(num1)
num2 = int(num2)
if operator == "+" :
    print(num1 + num2)
elif operator == "-" :
    print(num1 - num2)
elif operator == "*" :
    print(num1 * num2)
elif operator == "/" :
    print(num1 / num2)
elif operator == "%" :
    print(num1 % num2)
else :
    print("Invalid numbers")
```


→ Range
point (list)

list = range(5)
(0, 5)

→ Loops

— while loop

i = 1

while i <= 5 :

print(i)

i = i + 1

i = 1

while i <= 5 :

print(i * "#")

i = i + 1

i = 5

while i >= 0 :

print(i * "#")

i = i - 1

— For loop

used for iterative item

for item in range(5):

print(item)

in python we access index from 0 to onward
and also 0 to backward means

-1, -2, ...

→ list Datatype (complex datatype)

— collection of items

marks = [34, 76, 91]

print(marks)

print(marks[1])

print(marks[-2])

For sub-list

print(marks[1:3])

starting index

ending index
whitespace not
include

For print individual element of list

in single line we use loop.

for score in marks:

print(score)

— Operation on list

1) Append operation

To add item, element

marks = [91, 94]

marks.append(99)

~~marks~~ [91, 94, 99]

print(marks)

2) Insert operation

marks.insert(0, 99)

index

value

print(marks)

~~marks~~ [99, 91, 94]

3) in (for check particular item exist in list)

```
marks = [91, 92, 93, 94]
```

```
print(91 in marks)
```

```
# True
```

4) length operation

```
print(len(marks))
```

```
# 4
```

If we want to iterate on list
we also use while loop with length op.

```
marks = [91, 92, 93, 94]
```

```
i = 0
```

```
while i < len(marks):
```

```
    print(marks[i])
```

```
    i = i + 1
```

5) For clear element in list

```
marks.clear()
```

```
print(marks)
```

```
# []
```


Break & Continue

```
Food = ["Apple", "Banana", "Grapes"]
```

```
for fruit in Food:
```

```
    if fruit == "Grapes":
```

```
        break
```

```
    print(fruit)
```

```
# Apple, Banana
```

:- stop loop while condition is true

```
if fruit == "Apple"
```

```
    continue
```

```
print(fruit)
```

```
# Banana, Grapes
```

Tuple (immutable) (can't change)

```
marks = (a1, a2, a3)
```

Operation on tuple

```
print(marks.count(a1))
```

```
# 1
```

```
print(marks.index(a3))
```

```
# 2
```

Note:- () Not compulsory

Set { }

→ Index is not exist in set (marks[0])

→ Un-order collection of element

```
marks = {91, 92, 91, 93, 91}
```

```
print(marks)
```

```
# 93, 92, 91
```

Dictionary

— Key value pair is store in it

```
marks = { "math" : 99, "Urdu" : 94 }
```

 ↓ ↓
 key value

```
print(marks["math"])
```

To add new marks

```
marks["physics"] = 97;
```

```
print(marks)
```

:- Value of marks can also be change

```
# math: 99, Urdu: 94, physics: 97
```


1. In-Built Functions
- : 2. Module Functions
3. User-defined Function

1

int(), str(), bool()

2

Module Function

related function and related variable
store in same file called module

we use

```
import math
```

use all function of math

```
print(dir(math)).
```

→ print all function
of math

For only particular function

```
from math import sqrt
```

```
print(sqrt(16))
```

(for all
*)

3

User-defined function

Syntax

def function_name (parameters):

def print_sum(n1, n2):

print (n1 + n2)

print_sum(1, 4)



Customer ID